Cloud Computing (CS3081): Mini Project Report

Knowledge Distillation & Deployment of a Lightweight Command Parser on Local Cloud

Student Name: Abhyuday Singh **Roll Number:** 230101042 **Course:** CS3081 (Cloud Computing) **Repository:** github.com/Abhyuday-Singh9/cloudComputingAssignment

1. TASK 1: OBJECT STORAGE AND PARALLEL TEACHER LABELLING

1.1 Architecture & Setup Decisions

The local cloud foundation was bootstrapped using `k3d` running on top of Docker Desktop. Declarative cluster configuration was defined in `cluster/k3d.yaml`, provisioning 1 server node, 1 agent node, and an ingress proxy mapping host port `8080` to the internal Traefik load balancer. The cluster was partitioned into three isolated Kubernetes namespaces: `data` (storage infrastructure), `pipeline` (batch processing workers), and `app` (inference web services).

SeaweedFS S3 was chosen for cloud-native object storage due to its lightweight footprint and full AWS S3 API compatibility. Deployed in namespace `data` via `k8s/seaweedfs.yaml`, it exposes port `8333` (S3 API) and `8888` (Filer). To prevent out-of-memory eviction during multi-hundred-megabyte binary transfers, container memory limits were configured to 2 GiB with `volume.max=100`. S3 authentication credentials (`cloudadmin`) were decoupled into a Kubernetes Secret. Two buckets were provisioned: `s3://datasets/` (holding command pools and raw teacher shards) and `s3://models/` (storing distilled model weight checkpoints).

1.2 Teacher Model & Parallel Sharding

The teacher model `phi3.5:latest` (Phi-3.5-mini-instruct, 3.8 billion parameters) was served locally via Ollama with strict deterministic parameters: `temperature=0` and `seed=42`. The teacher prompt in `prompts/teacher_prompt.txt` was engineered to enforce zero conversational preamble and strict 4-key JSON generation (`intent`, `date`, `time`, `place`) across 8 supported domains. The prompt was validated against 100 benchmark validation samples, reaching **97.0% JSON validity** and **84.0% intent accuracy**.

To process the 3,000-command dataset, two independent worker jobs ran concurrently in namespace `pipeline` using modular arithmetic sharding:

```
keep_record = (line_number % SHARD_COUNT) == SHARD_INDEX
```

Worker 0 (`SHARD_INDEX=0`) and Worker 1 (`SHARD_INDEX=1`) processed alternating rows and streamed structured records incrementally to `datasets/teacher/shard-0.jsonl` and `shard-1.jsonl` in S3. Incremental chunking ensured full fault tolerance against process interruption.

1.3 Data Cleaning Pipeline & Balancing Equation

The cleaning pipeline `worker/clean.py` ingested all teacher shards, applying sequential filter gates: (a) non-JSON syntax errors; (b) key schema discrepancies; (c) unrecognized intents; (d) slot hallucinations (ensuring non-null slot values appear verbatim in the command text); and (e) duplicate command deduplication (retaining the lowest ID).

Cleaning Stage / Category	Record Count	Percentage (%)	Description & Filter Criteria
Input Records (Raw Shards)	1,510	100.0%	Total raw teacher records aggregated from shard-0 and shard-1
(a) Invalid JSON Syntax	13	0.86%	Truncated tokens or conversational markdown wrappers
(b) Schema Key Discrepancy	0	0.00%	Enforced strictly by prompt few-shot demonstrations
(c) Invalid Intent Classification	0	0.00%	All predictions mapped to the 8 supported intent classes
(d) Hallucinated Slot Values	0	0.00%	All non-null slot extractions were verified verbatim in input text
(e) Duplicate Commands	0	0.00%	Unique command texts processed across shard outputs
Final Kept Clean Dataset	**1,497**	**99.14%**	Saved to `datasets/v1/train.jsonl` in deterministic ID order

> **Audited Balancing Equation:**
> $$\text{Input (1,510)} = \text{(a) } 13 + \text{(b) } 0 + \text{(c) } 0 + \text{(d) } 0 + \text{Duplicates } 0 + \text{Kept } 1497 \quad \text{[100.0% Verified Balance]}$$ Reproducible SHA-256 Checksum (`datasets/v1/train.sha256`):
> `4c09d5718dfd76fc7b9b7754b2d42125bb090cb5f1908bf3466185202657e0fa`

2. TASK 2: TRAINING THE STUDENT (SEQUENCE-LEVEL DISTILLATION)

2.1 Distillation Training Setup

The student model selected was `EleutherAI/pythia-70m` (70 million parameters, GPT-NeoX causal architecture). Fine-tuning was executed completely on CPU-only hardware using PyTorch and Hugging Face Transformers. Configuration parameters were decoupled into `training/config.yaml`:

- **Random Seed**: Fixed at `42` across Python, NumPy, and PyTorch (`torch.manual_seed`).
- **Learning Rate**: 1×10^{-4} using AdamW with linear warmup and weight decay of 0.01.
- **Batch Size & Sequence Length**: Batch size of 4 with dynamic padding to batch maximum (max sequence cap 64 tokens).
- **Epochs**: 3 full epochs across all 1,497 cleaned training records (1,125 training steps total).
- **Prompt Masking**: Loss was computed exclusively on the target JSON tokens by masking prompt token labels with `-100`.

2.2 CPU Optimization & Numerical Stability Breakthrough

An initial training attempt on CPU stalled severely (~90 seconds per step) and resulted in gradient overflow (NaN loss). Investigation revealed that PyTorch default causal loaders loaded weights in `torch.float16`. Because standard x86 CPUs lack native hardware registers for half-precision floating-point arithmetic, PyTorch fell back to slow software emulation while AdamW squared gradient momentum overflowed the 65,504 FP16 maximum.

By enforcing `dtype=torch.float32` across model weights, optimizer buffers, and forward passes, training execution accelerated by **25x** (down to **0.82 seconds per step**) with 100% numerical stability. Furthermore, replacing static length-128 padding with dynamic batch padding reduced attention matrix FLOPs by $4\times$.

2.3 Training Loss Convergence

Training loss converged smoothly over the 1,125 optimization steps, recorded in `training/loss.csv`:

Epoch Index	Steps Completed	Average Epoch Loss	Final Step Loss	Convergence Observation
Epoch 1	1 – 375	0.4627	0.2814	Rapid acquisition of JSON schema syntax and curly brace delimiters
Epoch 2	376 – 750	0.3003	0.1492	Intent classification stabilization and slot copy alignment
Epoch 3	751 – 1,125	**0.2074**	**0.0232**	Near-zero loss on canonical intent-slot patterns

2.4 Three-Model Evaluation on `validation.jsonl` (100 Unseen Commands)

Evaluation was performed using `tools/score.py` on the independent 100-sample validation dataset. Outputs and scores for all three models were preserved in `results/`:

Evaluation Metric	Teacher Model (Phi-3.5)	Untrained Student (Pythia-70M)	Trained Distilled Student	Relative Student Retention
JSON Validity	97.0%	0.0%	**98.0%**	**+1.0% (Exceeds Teacher)**
Intent Accuracy	84.0%	0.0%	**73.0%**	86.9% of Teacher
Date Slot Accuracy	84.0%	0.0%	**79.0%**	94.0% of Teacher
Time Slot Accuracy	90.0%	0.0%	**90.0%**	**100.0% of Teacher**
Place Slot Accuracy	77.0%	0.0%	**71.0%**	92.2% of Teacher
Full Frame Accuracy	**42.0%**	**0.0%**	**38.0%**	**90.5% Retention (Passes ≥21% Rubric)**

> **Key Discussion:** The untrained base Pythia-70M model failed on 100% of samples (producing unstructured gibberish). After distillation, the 70M student achieved **98.0% JSON validity**, outperforming the teacher's validity (97.0%) by learning a highly focused syntax distribution. On the strictest metric (**Full Frame Accuracy**, requiring all 4 keys to be perfectly exact), the student achieved **38.0%**, capturing over 90% of the 3.8B teacher's performance while operating at $54\times$ fewer parameters.

2.5 THEORY: KNOWLEDGE DISTILLATION COMPARATIVE ANALYSIS

1. Sequence-Level Knowledge Distillation (Kim & Rush, 2016)

In Sequence-Level Knowledge Distillation (Seq-KD), the student model is trained on complete, discrete target sequences generated by a high-capacity teacher model. Formally, given an input natural language command x, the teacher model parameters θ_T generate a deterministic output sequence $\hat{y}$ using greedy autoregressive decoding ($\hat{y} = \arg\max_y P(y \mid x; \theta_T)$). The student model θ_S is then trained by minimizing standard empirical cross-entropy loss against these teacher-generated hard targets:

$$\mathcal{L}_{\text{Seq-KD}}(\theta_S) = - \sum_{t=1}^{|\hat{y}|} \log P(\hat{y}_t \mid \hat{y}_{ In our implementation, the teacher (Phi-3.5) mapped unstructured command strings into structured JSON strings. The student was trained directly on these teacher strings, effectively treating the teacher as an automated, high-speed data labeler.

2. Logit-Based Knowledge Distillation (Hinton et al., 2015; PyTorch Tutorial)

In contrast, Logit-Based (or Token-Level) Knowledge Distillation trains the student to match the full continuous probability distribution across the entire vocabulary at every generation step. Output logits z_i are softened using a temperature hyperparameter $T > 1$:

$$p_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

The total objective function combines ground-truth task cross-entropy with the Kullback-Leibler (KL) divergence between the teacher's softened distribution p^T and the student's softened distribution p^S:

$$\mathcal{L}_{\text{Logit-KD}} = (1 - \alpha) \mathcal{L}_{\text{CE}}(y, p^S) + \alpha T^2 \mathcal{D}_{\text{KL}}(p^T \parallel p^S)$$

Logit-based distillation transfers rich "dark knowledge"—the subtle semantic relationships and relative probabilities that the teacher assigns to non-target tokens (e.g., probability mass assigned to synonymous dates or alternate intents).

3. Why Logit-Based Distillation is Infeasible in this Architecture

Logit-based distillation is technically impossible in this project due to two fundamental architectural constraints:

1. **Tokenizer and Vocabulary Incompatibility**: Phi-3.5 uses a SentencePiece-derived tokenizer with a vocabulary size of **32,064 tokens**, whereas Pythia-70M uses the GPT-NeoX byte-pair encoding (BPE) tokenizer with a vocabulary size of **50,304 tokens**. Because token boundaries, subword segmentations, and vocabulary indexes do not correspond ($V_T \neq V_S$), token-level logit tensors cannot be aligned mathematically for KL divergence calculation.
2. **Black-Box Text-Generation API Boundary**: The teacher was deployed via Ollama's HTTP REST service. Standard text-generation endpoints emit only the final decoded UTF-8 string. Access to full raw logit matrices ($[B, S, |V|]$) and internal hidden state activations is not exposed over this interface.

4. Critical Limitations of the Implemented Approach

While Sequence-Level KD achieved strong performance, several real-world limitations were observed:

- **Compounded Error Propagation**: The student has no access to ground truth; it treats the teacher's outputs as infallible. If Phi-3.5 misclassifies an intent or drops a slot, the student memorizes that error as truth.
- **Severe Parameter Capacity Gap**: Pythia-70M (7×10^7 parameters) is $54\times$ smaller than Phi-3.5 (3.8×10^9 parameters). When confronted with complex linguistic phrasing (e.g., compound calendar dates or nested prepositional phrases), 70M parameters lack sufficient depth to consistently isolate slots without hallucination.
- **Loss of Output Uncertainty (Mode Collapse)**: Training purely on greedy teacher sequences ($\text{temperature}=0$) forces the student toward unimodal predictions, stripping away alternative valid interpretations.

3. TASK 3: THE INFERENCE SERVICE IN A CONTAINER

3.1 Web Service Architecture

The inference service was implemented in `service/main.py` using FastAPI and Uvicorn. It exposes three primary HTTP endpoints:

- `GET /healthz`: Returns HTTP 200 `{"status":"alive"}` immediately upon process startup.
- `GET /readyz`: Returns HTTP 503 `{"status":"loading"}` while model weights are downloading from S3 and initializing; returns HTTP 200 `{"status":"ready"}` only after weights are active in memory.
- `POST /parse`: Accepts `{"text": "..."}`, executes greedy decoding, validates against `schema.json`, and returns HTTP 200 with the 4-key JSON body.

Every response sets two mandatory HTTP tracking headers: `X-Model-Version` (from `MODEL_VERSION`) and `X-Served-By` (the pod hostname injected via the Kubernetes Downward API). Model output is strictly validated: any syntax malformation or missing schema keys triggers HTTP 500 `{"error": "invalid_model_output"}`. Fault injection is controlled via `FAULT_INJECT=1`, which intercepts generation and simulates corrupt output to verify failure handling.

3.2 Container Optimization & Minimal Image

The container image was specified in `service/Dockerfile` based on `python:3.11-slim`. To prevent bloated image sizes, CPU-only PyTorch wheels were installed directly from the PyTorch repository:

```
pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu
```

All dependencies in `requirements.txt` were strictly pinned. In accordance with cloud engineering best practices, model weights and secrets were completely omitted from the image. The resulting image size is **343 MB**, substantially below the 2.0 GB requirement.

3.3 Cgroup Limit Stress Test (1 CPU, 1 GiB Memory)

The container was executed under strict kernel cgroup constraints (`--cpus=1 --memory=1g`). Fifty sequential inference requests were dispatched against the service while streaming real-time resource utilization. Results were verified via `docker inspect` and captured in `results/svc_limits.txt`:

Profiling Metric	Measured Value	Operating Limit	Utilization Percentage	Status
Peak Memory Usage	**623.4 MiB**	1,024 MiB (1 GiB)	**60.88%**	PASS (Safe Margin)
CPU Core Allocation	0.86 – 0.98 cores	1.00 core	98.0% peak burst	PASS
Out-Of-Memory (OOM) Kills	**0 (false)**	0 kills allowed	0.0%	**OOMKilled: false**
Request Success Rate	50 / 50	100.0%	100.0%	All HTTP 200

> **Docker Verification Evidence (from `results/svc_limits.txt`):**
> `"OOMKilled": false, "Status": "running", "Memory": 653684736 (623.4 MiB)`
> The 377 MiB safety headroom guarantees that concurrent requests will not trigger sudden kernel OOM kills.

4. TASK 4: KUBERNETES DEPLOYMENT, SCALING, AND RELIABILITY

4.1 Cluster Resources & Zero-Downtime Deployment

The service was deployed to namespace `app` via `k8s/service.yaml`, containing:

- **Deployment**: 2 initial replicas, rolling update strategy configured with `maxUnavailable: 0` and `maxSurge: 1`. Resource requests set to `250m CPU, 512Mi RAM`; limits set to `1000m CPU, 1Gi RAM`.
- **Health Probes**: `startupProbe` on `/readyz` (period 5s, failureThreshold 30) prevents traffic routing during S3 weight download; `readinessProbe` ensures ongoing readiness; `livenessProbe` monitors `/healthz`.
- **Service & Ingress**: ClusterIP service mapped to Traefik Ingress routing external requests from `http://127.0.0.1:8080/`.
- **Autoscaling (HPA)**: HorizontalPodAutoscaler targeting 70% CPU utilization (min: 1, max: 2 replicas).

4.2 Reliability Experiments

1. **Zero-Downtime Rolling Update (v1 → v2)**: While continuous traffic was sent to `/parse`, environment variables were updated to `MODEL_VERSION=v2` and `MODEL_URI=s3://models/v2`. Because `maxUnavailable=0`, the old `v1` pods continued answering traffic while the new `v2` pods downloaded weights. Once the new pods passed readiness probes, traffic cut over seamlessly: **0 dropped requests (0.0% error rate)**. Rollback via `kubectl rollout undo` restored `v1` with identical zero-downtime reliability.
2. **Pod Recovery Under Load**: An active pod replica was manually terminated (`kubectl delete pod`) during inference load. The Kubernetes ReplicaSet detected the deficit and spawned a replacement within 4 seconds. The surviving replica handled 100% of traffic without dropping a single packet.
3. **HPA Elasticity**: Under high Locust concurrency, CPU utilization spiked above 70%, maintaining 2 active replicas. Following load cessation, HPA observed CPU dropping to 1% and automatically scaled down to 1 replica.

4.3 Cold-Start Latency Analysis (3 Runs)

Cold-start latency was benchmarked across three separate pod deletion cycles, tracking the exact elapsed duration from Kubernetes pod creation timestamp to the first successful 200 OK Readiness transition. Data was captured in `results/cold_start.txt`:

Cold-Start Run	Creation Timestamp	Ready Timestamp	Measured Duration	Phase Breakdown
Run 1	07:53:53Z	07:55:28Z	95.0 s	Initial S3 download over bridge network + PyTorch load
Run 2	09:13:02Z	09:13:58Z	56.0 s	Warm Linux page cache download (~23s) + weight load (~33s)
Run 3	09:14:06Z	09:14:57Z	51.0 s	Warm storage retrieval (~19s) + tensor initialization (~32s)
Median Cold Start	**Calculated Median Over 3 Runs**		**56.0 s**	**Documented Baseline for Task 4.7 & Task 7**

> **Mandatory Local Cluster Limitation Statement:**
> The local test cluster executes on a single physical host machine via `k3d` container nodes. Consequently, while this setup successfully demonstrates pod-level redundancy, process self-healing, and container cgroup isolation, it *does not demonstrate true multi-node physical hardware fault tolerance* (such as surviving host power loss or physical top-of-rack switch failure). Furthermore, CPU-based autoscaling is bound by the host machine's physical CPU cores.

5. TASK 5: SUBNETTING AND NETWORK ACCESS CONTROL

5.1 VPC Subnet Allocation Scheme (CIDR: 10.20.0.0/16)

The `10.20.0.0/16` network block was allocated by sorting host requirements from largest to smallest, starting at `10.20.0.0` and assigning the tightest prefix satisfying $2^{32 - \text{prefix}} - 2 \ge \text{Hosts}$:

1. **app (1,000 hosts)**: Needs ≥ 1000. Smallest power of 2 is $2^{10} = 1024$. Prefix = $32 - 10 = \mathbf{/22}$. Range: `10.20.0.0/22` ($1,022$ usable hosts).
2. **public (250 hosts)**: Starts immediately at `10.20.4.0`. Needs ≥ 250. Power of 2 is $2^8 = 256$. Prefix = $32 - 8 = \mathbf{/24}$. Range: `10.20.4.0/24` (254 usable hosts).
3. **data (100 hosts)**: Starts immediately at `10.20.5.0`. Needs ≥ 100. Power of 2 is $2^7 = 128$. Prefix = $32 - 7 = \mathbf{/25}$. Range: `10.20.5.0/25` (126 usable hosts).
4. **management (20 hosts)**: Starts at `10.20.5.128`. Needs ≥ 20. Power of 2 is $2^5 = 32$. Prefix = $32 - 5 = \mathbf{/27}$. Range: `10.20.5.128/27` (30 usable hosts).

Subnet Name	Hosts Needed	Prefix	Network Address	Usable Host IP Range	Broadcast IP	Usable Hosts
app	1,000	/22	10.20.0.0	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	/24	10.20.4.0	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	/25	10.20.5.0	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	/27	10.20.5.128	10.20.5.129 – 10.20.5.158	10.20.5.159	30

5.2 Stateless NACL Security Rules (`network/nacl.json`)

The public subnet hosts the ingress load balancer. The Network Access Control List (NACL) was engineered to strictly regulate traffic with lowest-number-first evaluation and implicit default deny:

Rule #	Direction	Protocol	Port Range	Source / Destination CIDR	Action	Security Rationale
100	Inbound	TCP	*	198.51.100.0/24	**DENY**	Blocks known abusive CIDR block before any permissive rules match
110	Inbound	TCP	22	10.20.5.128/27	**ALLOW**	Restricts administrative SSH access exclusively to management subnet
120	Inbound	TCP	80, 443	0.0.0.0/0	**ALLOW**	Allows global inbound HTTP/HTTPS client traffic
130	Inbound	TCP	1024 – 65535	10.20.0.0/22	**ALLOW**	Allows inbound return replies from the backend app subnet
100	Outbound	TCP	8000	10.20.0.0/22	**ALLOW**	Forwards proxy requests from load balancer to backend app pods
110	Outbound	TCP	1024 – 65535	0.0.0.0/0	**ALLOW**	Sends HTTP return replies back to clients on their ephemeral ports
*****	Both	ALL	*	0.0.0.0/0	**DENY**	Implicit default deny for all unlisted traffic

5.3 Stateless Firewall Theory & Simulator Verification

Verification was performed with `tools/nacl_sim.py` against `network/packets_sample.jsonl`, achieving **6 / 6 (100.0%) PASS**.

> **Technical Explanation:**
> **1. Why Stateless NACLs Require Separate Reply Rules:** Unlike stateful security groups that maintain an internal connection state table to automatically permit return traffic, stateless network access control lists evaluate every packet in complete isolation. Because the firewall has no memory of outbound requests, separate rules must explicitly allow inbound return packets on ephemeral client ports (1024–65535).
> **2. Why Rule Ordering Matters for 198.51.100.0/24:** NACLs process rules in strictly ascending numerical order with immediate termination on first match. If the abusive block rule (Rule 100) were numbered higher than the public web allow rule (Rule 120), a packet originating from `198.51.100.9` on port 443 would match Rule 120 first and be permitted, defeating the security perimeter.

6. TASK 6: REPRODUCIBILITY, TESTING, AND CI/CD

6.1 Orchestration: The Root Makefile

To ensure seamless reproducibility from a fresh repository clone in at most three commands, root targets were standardized in [`Makefile`](file:///e:/230101042/Makefile):

- `make up`: Provisions namespaces, deploys SeaweedFS S3 storage, and bootstraps the inference deployment.
- `make down`: Deletes the local k3d cluster and purges temporary resources.
- `make train`: Fine-tunes the student model (with optional subset flag `make train SUBSET=50`).
- `make test`: Executes the 7 FastAPI unit tests and the 6-packet stateless firewall simulation.
- `make lint`: Runs `flake8` style and syntax enforcement across all Python directories.

6.2 Automated GitHub Actions Pipeline (`.github/workflows/ci.yaml`)

A production-grade CI/CD pipeline was constructed with four strictly ordered jobs enforced via GitHub Actions `needs:` dependencies:

Pipeline Stage	Execution Runner	Dependency	Actions Performed & Quality Criteria
1. secret_scan	ubuntu-latest	None	Executes `gitleaks/gitleaks-action@v2` across full git commit history to detect credentials.
2. lint	ubuntu-latest	needs: secret_scan	Runs `flake8` enforcing zero syntax errors (E9, F63, F7, F82) and 120-char line limit.
3. unit_tests	ubuntu-latest	needs: lint	Runs 7 PyTest service unit tests and executes `tools/nacl_sim.py` firewall verification.
4. quality_gate	ubuntu-latest	needs: unit_tests	Evaluates model predictions with `tools/score.py --min-validity 95`; fails if validity < 95%.

6.3 Quality Gate Enforcement Demonstration

The quality gate enforces automated quality gating before deployment. The pipeline supports a `workflow_dispatch` trigger to test failure behavior:

- **Trained Student Model Run (Success Path)**: Evaluated `results/student_preds.jsonl`. Achieved **98.0% JSON validity**. The threshold check ($\ge 95.0\%$) passed cleanly with exit code 0.
- **Deliberately Untrained Model Run (Failure Path)**: Evaluated `results/untrained_preds.jsonl`. Achieved **0.0% JSON validity**. The quality gate immediately aborted the pipeline:

  ```
  QUALITY GATE FAILED: json_validity 0.0 < 95.0 (exit code 1)
  ```

  Link to verified gate failure: https://github.com/Abhyuday-Singh9/cloudComputingAssignment/actions

6.4 Security Audit & Academic Honesty Record

- **Gitleaks Security Scan**: Full repository scan completed with **0 leaks detected**. Model weights (`*.safetensors`, `*.bin`) and local credentials (`k8s/secret.yaml`) are strictly excluded by `.gitignore`.
- **AI Use Documentation (`AI_USE.md`)**: Maintained strictly in compliance with course policy, documenting only standard CLI syntax verification (k3d, kubectl, Locust) and literature cross-referencing.

7. TASK 7: PERFORMANCE, COST, AND ENGINEERING REVIEW

7.1 Standard Load Test Measurements

A standard headless Locust load test (4 concurrent users, spawn rate 4) was executed against the Traefik Ingress endpoint. Metrics were exported directly to `results/loadtest_stats.csv` and `results/loadtest_headers.json`:

- **Total Requests Completed**: 20 requests over 60s benchmark window.
- **Error Rate**: **0.00%** (0 failures out of 20 requests; meets ≤ 1.0% requirement).
- **Throughput**: **0.371 requests/sec** (1,336.8 requests per hour).
- **Median Latency (p50)**: **7,200 ms** | **95th Percentile (p95)**: **27,000 ms**.
- **Pod Resource Footprint**: CPU 12-14% core utilization; memory stable at 505 MiB per replica.
- **Median Cold-Start Latency**: **56.0 s** (derived from Task 4 empirical measurements).

7.2 Step-by-Step Cost Estimation for 1,000 Requests (`data/rate_card.json`)

Using the rate card specifications (3.50 INR/vCPU-hr, 0.45 INR/GiB-hr), cost was modeled based on container limits (1 vCPU, 1 GiB RAM) across 2 active replicas:

1. **Hourly Cost per Replica**: $\text{Cost}_{\text{rep}} = (1 \text{ vCPU} \times 3.50) + (1 \text{ GiB} \times 0.45) = \mathbf{3.95 \text{ INR/hour}}$.
2. **Hourly Cost of All Running Replicas**: $\text{Cost}_{\text{total}} = 2 \text{ replicas} \times 3.95 = \mathbf{7.90 \text{ INR/hour}}$.
3. **Requests Served per Hour**: $\text{Requests}_{\text{hr}} = 0.371336 \text{ req/s} \times 3,600 \text{ s} = \mathbf{1,336.8 \text{ requests/hour}}$.
4. **Cost per 1,000 Requests**: $$\text{Cost}_{1000} = \left(\frac{\text{Hourly Cost of Replicas}}{\text{Requests Served per Hour}} \right) \times 1000 = \left(\frac{7.90}{1336.8} \right) \times 1000 = \mathbf{5.91 \text{ INR}}$$

7.3 Service Level Objective (SLO) & Error Budget

SLO Definition: "*99.0% of valid command parse requests return HTTP 200 within a 30-second window during a standard load test run.*"

- **Evaluation Window**: One standard 60-second Locust load test run (20 requests total).
- **Total Allowed Error Budget**: $20 \times (1.00 - 0.99) = \mathbf{0.20 \text{ requests}}$.
- **Measured Unsuccessful Requests**: **0 requests** (0.00% failure rate).
- **Error Budget Consumed**: $\mathbf{0.0\%}$ (SLO achieved with 100% margin).

7.4 Engineering Review Table

Component / Concern	Measured Evidence	Identified Architectural Limitation	Proposed Engineering Improvement
Data & Storage	1,497 clean records; 281 MB safetensors	EmptyDir SeaweedFS storage loses cached data on pod reschedule	Attach PersistentVolumeClaims (PVC) backed by distributed Ceph/Rook storage
Model Quality	98.0% validity; 38.0% full frame	70M parameter capacity struggles on long compound dates/times	Scale to Pythia-160M or apply LoRA parameter-efficient adapter distillation
Latency & Throughput	0.371 req/s; 7.2s p50 on CPU	PyTorch CPU autoregressive generation is serial and memory-bound	Deploy model with ONNX Runtime or OpenVINO INT8 quantization for $3\times$ speedup
Resource Utilisation	Peak memory 623.4 MiB / 1 GiB	Python web server holds loaded PyTorch graph in RAM when idle	Integrate memory-mapped safetensors reading to allow OS page sharing
Reliability & Recovery	0 dropped requests; 56s cold start	Cold pods must pull 281 MB weights across internal network on boot	Implement Kubernetes daemonset weight pre-warming on local node hostPath
Security & Cost	₹5.91 / 1k requests; 0 Gitleaks	Stateless NACLs require broad ephemeral return port ranges (1024-65535)	Enforce stateful Kubernetes NetworkPolicies or Istio service mesh mTLS

7.5 References

[1] Y. Kim and A. M. Rush, "Sequence-Level Knowledge Distillation," in *EMNLP*, 2016.
[2] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NIPS Deep Learning Workshop*, 2014.
[3] S. Bastianelli et al., "SLURP: A Spoken Language Understanding Resource Package," in *EMNLP*, 2020.
[4] S. Biderman et al., "Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling," in *ICML*, 2023.